English LaTeX Translation and Consolidation of the MMHCL-Plus Notes

Spectral Radius Tracking, Mutual Information Diagnostics, and Q1-Style Ablation Design

1. Context

Based on a careful analysis of the GitHub codebase, including `TwoStageTrainer (Rev5.2)`, `LayerwiseEncoder` with `max_g_layers=2`, `build_hypergraph_laplacian` with

$$\Theta = D_v^{-1/2} H W_e D_e^{-1} H^T D_v^{-1/2},$$

`svd_filter_incidence`, and the configuration `MMHCLPlusConfig` with five tasks and $D = 4096$, the following implementation package is proposed for Spectral Radius Tracking and MINE/KSG Mutual Information Estimation as a DPI sanity check.

This consolidated document is written to improve coherence and cohesion relative to the raw notes. It organizes the material into a theoretical foundation, diagnostic modules, ablation variants, notebook integration, and statistical validation.

2. Theoretical foundation

2.1. Why these two tools are needed

The Data Processing Inequality (DPI) states that if

$$X \to h^{(l)} \to h^{(l+1)}$$

forms a Markov chain, and each GNN layer is deterministic, then

$$I(X; h^{(l)}) \geq I(X; h^{(l+1)}).$$

In other words, mutual information (MI) should decrease monotonically across layers.

At the same time, the spectral radius $\lambda_{\max}(\Theta)$ of the diffusion operator Θ controls the rate of oversmoothing. When $\lambda_{\max}(\Theta)$ approaches 1, node embeddings drift toward the stationary distribution, which is precisely the oversmoothing regime. Neighbor-Layer Contrastive Learning (Neighbor-Layer CL) is intended to counteract this effect by preserving a meaningful separation between $h^{(l)}$ and $h^{(l+1)}$.

2.2. Sanity-check criteria

If the architecture is functioning as intended, the following sanity checks should hold:

- Mutual information should decrease monotonically across layers, consistent with DPI.
- The spectral radius should remain below 1 and should decrease when Dirichlet regularization is enabled.
- Neighbor-Layer CL should slow the rate of MI decay relative to a baseline without NLCL.

3. Diagnostic package structure

3.1. New file: `codes/mmhcl_plus/diagnostics/__init__.py`

```
from .spectral_radius import (
    compute_spectral_radius,
    track_spectral_radius_per_epoch,
    SpectralRadiusTracker,
)
from .mutual_information import (
    MINEEstimator,
    ksg_mutual_information,
    LayerwiseMITracker,
)
```

3.2. New file: `codes/mmhcl_plus/diagnostics/spectral_radius.py`

The purpose of this module is to track $\lambda_{\max}(\Theta)$, where $\Theta = I - L$ is the diffusion operator derived from the hypergraph Laplacian built by `build_hypergraph_laplacian()`.

The key architectural insight in Rev5.2 is the following:

- $L = I - \Theta$, so the eigenvalues of Θ are $1 - \lambda_i(L)$.
- $\lambda_{\max}(\Theta)$ controls the rate of oversmoothing.
- Dirichlet energy regularization pushes the eigenvalues of L away from 0, which is equivalent to pulling $\lambda_{\max}(\Theta)$ away from 1.

A typical usage pattern is:

```
tracker = SpectralRadiusTracker(interaction_matrix)
for epoch in range(n_epochs):
    # training step
    tracker.log(epoch)
tracker.plot()
tracker.to_dataframe()
```

3.3. Core functions in `spectral_radius.py`

- `compute_spectral_radius(laplacian, method="arpack", tol=1e-6)` computes the spectral radius of Θ without explicitly forming Θ for large sparse graphs.
- `compute_spectral_gap(laplacian, tol=1e-6)` computes $\lambda_2(L)$, the second-smallest eigenvalue of L, as a measure of connectivity and smoothing behavior.
- `compute_layer_spectral_radius(layer_embeddings)` estimates an empirical transition map between consecutive layer embeddings and computes the corresponding spectral contraction factor.
- `track_spectral_radius_per_epoch(interaction_matrix, svd_top_k=10)` performs a one-shot comparison of the spectral radius before and after SVD filtering.

3.4. Class: `SpectralRadiusTracker`

This tracker is intended for direct integration into the training loop. It stores two kinds of history:

- Laplacian-based history, including spectral radius and spectral gap per epoch.
- Layer-based history, including empirical contraction factors between consecutive hidden layers.

The class provides methods such as `log_laplacian()`, `log_layer_contraction()`, `update_laplacian()`, `to_dataframe()`, and `plot()` for analysis and visualization.

4. Q1-style ablation strategy

4.1. Ablation rationale

According to the revision-52 design notes, the MMHCL-Plus architecture should be evaluated through three complementary ablation groups in order to estimate the marginal contribution of each component:

- **A-series**: component-level ablations that switch individual modules on or off.
- **B-series**: hyperparameter sensitivity studies, including g-layers, projector dimension, ramp behavior, and hard-negative delay.
- **C-series**: comparisons among alternative loss balancers.

4.2. Proposed ablation matrix

ID	Variant	Description	Scientific goal
A0	Full (MMHCL-Plus)	All five losses, Hybrid Balancer, warmup ramp, delayed FAISS, VICReg with $D = 4096$, and SVD spectral augmentation	Upstream reference baseline
A1	w/o Neighbor-Layer CL	Disable $\mathcal{L}_{u2u}$ and $\mathcal{L}_{i2i}$, effectively reverting toward the original MMHCL	Measure the contribution of the NLGCL principle
A2	w/o SVD Spectral Aug	Set `apply_svd_filtering=False`	Quantify the contribution of spectral debiasing against popularity bias
A3	w/o VICReg Expansion	Reduce the projector dimension to 1024	Test the "Dimensionality Paradox" claim
A4	w/o CL Warmup Ramp	Activate contrastive learning abruptly at warmup	Measure the effect of CL activation shock

ID	Variant	Description	Scientific goal
A5	w/o Delayed FAISS	Use hard negatives from epoch 0	Validate the motivation for a delay of 50 epochs
A6	w/o Hutchinson Dirichlet	Disable $\mathcal{L}_{Dir}$	Measure the anti-oversmoothing contribution of the Dirichlet term
A7	w/ Ego-Final Anchor	Re-enable $\mathcal{L}_{ego_final}$, yielding 6 tasks	Validate the removal of the ego-final anchor in Rev5.2
A8	w/o Soft BYOL Cross	Disable $\mathcal{L}_{align}$	Measure the contribution of cross-branch alignment
B1–B3	$g \in \{1, 2, 3\}$	Sweep neighbor-layer depth	Validate the DPI-inspired safe zone $g \in \{1, 2\}$
C1	Uncertainty only	Kendall-style uncertainty weighting	Baseline loss balancer
C2	GradNorm only	Chen-style gradient normalization	Baseline loss balancer
C3	Fixed weights	Set all task weights to 1.0	Lower-bound baseline with no balancing

5. New file: `mmhcl_plus/ablation/ablation_config.py`

This file serves as the central ablation registry of the repository.

```python
# mmhcl_plus/ablation/ablation_config.py
"""
Ablation configuration registry for MMHCL-Plus (revision 52).
Each AblationVariant toggles a specific architectural component
to measure its marginal contribution on Recall@K / NDCG@K / Precision@K.
"""
from dataclasses import dataclass, field, asdict
from typing import Dict, List, Optional
import json, copy

@dataclass
class AblationVariant:
    name: str
    enable_neighbor_layer_cl: bool = True
    enable_svd_spectral:      bool = True
    vicreg_projector_dim:     int  = 4096
    vicreg_hidden_dim:        int  = 1024
    enable_cl_warmup_ramp:    bool = True
    ramp_epochs:              int  = 20
    enable_delayed_faiss:     bool = True
    delay_hard_negs_epoch:    int  = 50
    enable_dirichlet:         bool = True
```

```
    enable_ego_final_anchor: bool = False
    enable_soft_byol_cross:  bool = True
    g_layers: int = 2
    svd_top_k: int = 10
    balancer_type: str = "hybrid"
    balancer_switch_epoch: int = 30
    warmup_epochs: int = 15
    n_tasks: int = 5
    notes: str = ""

    def to_dict(self): return asdict(self)
    def dump(self, path): json.dump(self.to_dict(), open(path, "w"), indent=2)

REGISTRY: Dict[str, AblationVariant] = {
    "A0_full":         AblationVariant(name="A0_full",
                                       notes="Full MMHCL-Plus (rev52)"),
    "A1_no_nlcl":      AblationVariant(name="A1_no_nlcl",
                                       enable_neighbor_layer_cl=False,
                                       n_tasks=3,
                                       notes="Remove Neighbor-Layer CL (back to MMHCL)
    "),
    "A2_no_svd":       AblationVariant(name="A2_no_svd",
                                       enable_svd_spectral=False,
                                       notes="Disable SVD spectral filtering"),
    "A3_small_proj":   AblationVariant(name="A3_small_proj",
                                       vicreg_projector_dim=1024,
                                       vicreg_hidden_dim=512,
                                       notes="Shrink VICReg D 4096->1024 (Dim. Paradox
    )"),
    "A4_no_ramp":      AblationVariant(name="A4_no_ramp",
                                       enable_cl_warmup_ramp=False,
                                       notes="Abrupt CL activation (expect shock)"),
    "A5_no_delay":     AblationVariant(name="A5_no_delay",
                                       enable_delayed_faiss=False,
                                       delay_hard_negs_epoch=0,
                                       notes="FAISS hard negatives from epoch 0"),
    "A6_no_dirichlet": AblationVariant(name="A6_no_dirichlet",
                                       enable_dirichlet=False,
                                       n_tasks=4,
                                       notes="Remove Hutchinson Dirichlet"),
    "A7_ego_final":    AblationVariant(name="A7_ego_final",
                                       enable_ego_final_anchor=True,
                                       n_tasks=6,
                                       notes="Re-enable ego-final anchor (paradox)"),
    "A8_no_cross":     AblationVariant(name="A8_no_cross",
                                       enable_soft_byol_cross=False,
                                       n_tasks=4,
                                       notes="Remove Soft BYOL cross-branch"),
    "B1_g1":           AblationVariant(name="B1_g1", g_layers=1,
                                       notes="Neighbor depth g=1"),
    "B2_g2":           AblationVariant(name="B2_g2", g_layers=2,
                                       notes="Neighbor depth g=2 (default)"),
```

```
    "B3_g3":            AblationVariant(name="B3_g3", g_layers=3,
                                        notes="Neighbor depth g=3 (beyond DPI safe zone
    )"),
    "C1_uncertainty":   AblationVariant(name="C1_uncertainty",
                                        balancer_type="uncertainty"),
    "C2_gradnorm":      AblationVariant(name="C2_gradnorm",
                                        balancer_type="gradnorm"),
    "C3_fixed":         AblationVariant(name="C3_fixed",
                                        balancer_type="fixed"),
}

def get(variant_name: str) -> AblationVariant:
    if variant_name not in REGISTRY:
        raise KeyError(f"Unknown ablation variant '{variant_name}'. "
                       f"Available: {list(REGISTRY)}")
    return copy.deepcopy(REGISTRY[variant_name])
```

6. New file: `mmhcl_plus/trainer/hybrid_balancer.py`

This module extends the balancer so that it can support the entire C-series of ablations: uncertainty weighting, GradNorm, fixed weights, and the default hybrid switch from uncertainty to GradNorm.

```
class HybridLossBalancer(nn.Module):
    """
    mode in {'hybrid','uncertainty','gradnorm','fixed'}
    - hybrid     : Uncertainty (e<switch) -> GradNorm (e>=switch)  [rev52 default]
    - uncertainty: Kendall et al. 2018 homoscedastic weighting
    - gradnorm   : Chen et al. 2018 adaptive gradient normalization
    - fixed      : w_k = 1.0 for all k
    """
    def __init__(self, num_tasks=5, mode="hybrid", switch_epoch=30, alpha=1.5):
        super().__init__()
        self.mode, self.switch_epoch, self.alpha = mode, switch_epoch, alpha
        self.log_vars   = nn.Parameter(torch.zeros(num_tasks))
        self.gn_weights = nn.Parameter(torch.ones(num_tasks))
        self.register_buffer("L0", torch.zeros(num_tasks))
        self._L0_set = False

    def _uncertainty(self, losses):
        prec = torch.exp(-self.log_vars)
        return (prec * losses + self.log_vars).sum()

    def _fixed(self, losses):
        return losses.sum()

    def _gradnorm(self, losses, shared_params):
        if not self._L0_set:
            self.L0.copy_(losses.detach())
            self._L0_set = True
        w = self.gn_weights
```

```
        weighted = (w * losses).sum()
        return weighted

    def forward(self, losses: torch.Tensor, shared_params=None, epoch: int = 0):
        if self.mode == "fixed":
            return self._fixed(losses)
        elif self.mode == "uncertainty":
            return self._uncertainty(losses)
        elif self.mode == "gradnorm":
            return self._gradnorm(losses, shared_params)
        elif self.mode == "hybrid":
            return self._uncertainty(losses) if epoch < self.switch_epoch \
                   else self._gradnorm(losses, shared_params)
        raise ValueError(self.mode)
```

7. Notebook insertion cells

The notes propose appending new cells to the end of `Local_Random_seeds_train_mmhcl_baby_colab_origina`

7.1. Cell [Ablation-1]: import and select a variant

```
# === ABLATION DRIVER CELL ===
import os, json, wandb, torch, numpy as np, random
from mmhcl_plus.ablation.ablation_config import REGISTRY, get
from mmhcl_plus.trainer.hybrid_balancer import HybridLossBalancer

ABLATION_NAME = os.environ.get("ABLATION", "A0_full")
SEED = 612550091
cfg = get(ABLATION_NAME)
print(f"[Ablation] Running variant = {cfg.name} | notes: {cfg.notes}")
print(json.dumps(cfg.to_dict(), indent=2))

random.seed(SEED); np.random.seed(SEED)
torch.manual_seed(SEED); torch.cuda.manual_seed_all(SEED)
torch.backends.cudnn.deterministic = True
```

7.2. Cell [Ablation-2]: patch the model and training pipeline

```
args.g_layers              = cfg.g_layers
args.warmup_epochs         = cfg.warmup_epochs
args.delay_hard_negs_epoch = cfg.delay_hard_negs_epoch if cfg.enable_delayed_faiss
    else 0
args.vicreg_hidden_dim     = cfg.vicreg_hidden_dim
args.vicreg_projector_dim  = cfg.vicreg_projector_dim
args.enable_svd_spectral   = cfg.enable_svd_spectral
args.svd_top_k             = cfg.svd_top_k

projector = ExpandedProjector(
    input_dim=64,
    hidden_dim=cfg.vicreg_hidden_dim,
```

```
    output_dim=cfg.vicreg_projector_dim
).cuda()

hybrid_balancer = HybridLossBalancer(
    num_tasks=cfg.n_tasks,
    mode=cfg.balancer_type,
    switch_epoch=cfg.balancer_switch_epoch
).cuda()
```

7.3. Execution convention

A single variant can be launched from the command line using:

```
ABLATION=A4_no_ramp python train.py --use_mmhcl_plus True
```

8. Ablation aggregation and marginal contribution

The notes also describe exporting a CSV summary of the ablation results and computing the marginal contribution of each variant relative to `A0_full`. In practice, after loading the results into a dataframe `df`, the baseline row is indexed by `A0_full`, and each metric column is converted into a delta column Δ with respect to that baseline.

This makes the final table suitable for direct insertion into a Q1-style paper, because the table reports both absolute performance and the degradation or improvement induced by each architectural intervention.

9. Statistical validation

For Q1-level reporting, the notes explicitly require paired significance testing between `A0_full` and every ablation variant over at least three random seeds.

```
from scipy.stats import ttest_rel, wilcoxon
# df_seeds: rows = seeds, cols = variants, values = NDCG@20
for v in df_seeds.columns.drop("A0_full"):
    t, p_t = ttest_rel(df_seeds["A0_full"], df_seeds[v])
    w, p_w = wilcoxon(df_seeds["A0_full"], df_seeds[v])
    print(f"{v:15s} | Delta={df_seeds['A0_full'].mean()-df_seeds[v].mean():+.4f} "
          f"| t-test p={p_t:.4f} | Wilcoxon p={p_w:.4f}")
```

The rationale is straightforward: mean performance alone is insufficient for a high-quality publication, whereas paired statistical tests demonstrate whether each ablation effect is robust across seeds.

10. Repository checklist

A coherent implementation workflow for the repository is as follows:

1. Add `mmhcl_plus/ablation/ablation_config.py` as the canonical registry.
2. Add or replace `mmhcl_plus/trainer/hybrid_balancer.py` with the extended balancer.

3. Copy the six ablation cells into the notebook while preserving the working BPR, VICReg, and InfoNCE pipeline with seed 612550091.
4. Run variants from the CLI using the `ABLATION` environment variable.
5. Aggregate all metrics into a CSV table and compute marginal deltas against `A0_full`.
6. Perform paired t-tests and Wilcoxon signed-rank tests over multiple seeds before reporting final conclusions.

11. Closing note

This English LaTeX version consolidates the remaining material visible in `paste.txt` into a more coherent document structure. It preserves the main intent of the original notes: to provide a reproducible diagnostic and ablation framework for MMHCL-Plus that is suitable for implementation in a GitHub repository and for reporting in a Q1-style academic manuscript.